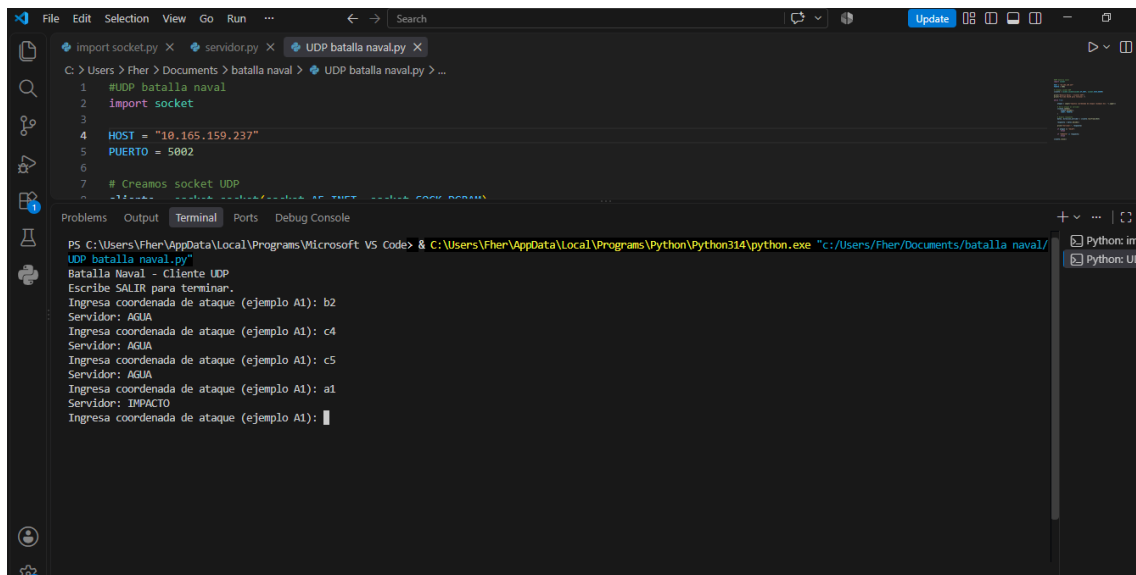


UDP batalla naval



```
import socket
servidor.py
UDP batalla naval.py
C:\Users\Fher\Documents> batalla naval > UDP batalla naval.py > ...
1 #UDP batalla naval
2 import socket
3
4 HOST = "10.165.159.237"
5 PUERTO = 5002
6
7 # Creamos socket UDP
8 sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

Problems Output Terminal Ports Debug Console

```
PS C:\Users\Fher\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Fher\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/Fher/Documents/batalla naval/UDP batalla naval.py"
Batalla Naval - Cliente UDP
Escribe SALIR para terminar.
Ingresa coordenada de ataque (ejemplo A1): b2
Servidor: AGUA
Ingresa coordenada de ataque (ejemplo A1): c4
Servidor: AGUA
Ingresa coordenada de ataque (ejemplo A1): c5
Servidor: AGUA
Ingresa coordenada de ataque (ejemplo A1): a1
Servidor: IMPACTO
Ingresa coordenada de ataque (ejemplo A1):
```



```
PROGRA EN RED — Python servidorudp.py — 119x22
fer@MacBook-Pro-de-Fer ~/Desktop/ICIC/AGO-DIC26/PROGRA EN RED % python3 servidorudp.py
Servidor UDP de Batalla Naval iniciado...
Esperando ataques...
Ataque recibido: B2
Desde: ('10.165.159.249', 59020)
Ataque recibido: C4
Desde: ('10.165.159.249', 59020)
Ataque recibido: C5
Desde: ('10.165.159.249', 59020)
Ataque recibido: A1
Desde: ('10.165.159.249', 59020)
```

En esta práctica se desarrolló una aplicación sencilla de Batalla Naval utilizando una arquitectura cliente-servidor en Python. Primero se implementó la comunicación mediante el protocolo TCP, donde el cliente se conecta al servidor, envía coordenadas de ataque y recibe como respuesta si el disparo fue “AGUA”, “IMPACTO” o si logró destruir todos los barcos. Posteriormente, se realizó una segunda versión utilizando el protocolo UDP, manteniendo la misma lógica del juego pero modificando la forma en que se envían y reciben los mensajes.

La práctica permitió observar de manera directa las diferencias entre TCP y UDP. En TCP se establece una conexión entre cliente y servidor antes de intercambiar información, mientras que UDP permite enviar datagramas sin establecer previamente una conexión permanente. También se trabajó con direcciones IP y puertos para lograr la comunicación entre dos equipos conectados a la misma red.

Además, se utilizó Git para llevar un control de versiones del proyecto, registrando los cambios realizados durante el desarrollo de las distintas implementaciones. De esta manera, la actividad permitió reforzar conceptos básicos de programación en red, sockets, protocolos de transporte y control de versiones.

```
PROGRA EN RED > servidorudp.py > ...
1  import socket
2
3  HOST = "0.0.0.0"
4  PUERTO = 5002
5  # Posiciones de los barcos
6  barcos = ["A1", "A2", "B4", "D3", "E5"]
7  # Creamos socket UDP
8  servidor = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
9  # Asociamos IP y puerto
10 servidor.bind((HOST, PUERTO))
11
12 print("Servidor UDP de Batalla Naval iniciado...")
13 print("Esperando ataques...")
14
15 while True:
16     # Recibe mensaje y dirección del cliente
17     datos, direccion_cliente = servidor.recvfrom(1024)
18
19     ataque = datos.decode().upper()
20
21     print("Ataque recibido:", ataque)
22     print("Desde:", direccion_cliente)
23
24     if ataque == "SALIR":
25         respuesta = "Juego terminado"
26         servidor.sendto(respuesta.encode(), direccion_cliente)
27         break
28
29     if ataque in barcos:
30
31         barcos.remove(ataque)
32
33         if len(barcos) == 0:
34             respuesta = "GANASTE - Todos los barcos fueron destruidos"
35         else:
36             respuesta = "IMPACTO"
37
38     else:
39         respuesta = "AGUA"
40
41     # Envía respuesta al cliente
42     servidor.sendto(respuesta.encode(), direccion_cliente)
43
44     if len(barcos) == 0:
45         break
46
47     servidor.close()
```

C: > Users > Fher > Documents > batalla naval > UDP batalla naval.py > ...

```
1  #UDP batalla naval
2  import socket
3
4  HOST = "10.165.159.237"
5  PUERTO = 5002
6
7  # Creamos socket UDP
8  cliente = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
9
10 print("Batalla Naval - Cliente UDP")
11 print("Escribe SALIR para terminar.")
12
13 while True:
14
15     ataque = input("Ingresa coordenada de ataque (ejemplo A1): ").upper()
16
17     # Envía ataque al servidor
18     cliente.sendto(
19         ataque.encode(),
20         (HOST, PUERTO)
21     )
22
23     # Espera respuesta
24     datos, direccion_servidor = cliente.recvfrom(1024)
25
26     respuesta = datos.decode()
27
28     print("Servidor:", respuesta)
29
30     if ataque == "SALIR":
31         break
32
33     if "GANASTE" in respuesta:
34         break
35
36 cliente.close()
```